

Министерство образования Республики Беларусь
Учреждение образования
Белорусский государственный университет информатики и
Радиоэлектроники

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №1
по курсу “Логические основы интеллектуальных систем”
Вариант 3

Выполнил:
Студент гр. 421702

Евик А.Н.

Проверил:

Ивашенко В.П.

Минск

2026

1. Грамматика сокращенного языка логики высказываний.....	3
2. Описание реализации.....	4
2.1. Структуры данных.....	4
2.2. Алгоритм.....	4
3. Схемы алгоритма.....	6
3.1 Схема алгоритма разбора строки.....	6
4. Тестирование программы.....	15
Вывод.....	17
Список использованных источников.....	18

Тема:

Программирование операций обработки и преобразований формул языка логики высказываний.

Цель:

Приобрести навыки программирования алгоритмов синтаксического разбора, интерпретаций и преобразований формул языка логики высказываний.

Задание:

Определить, является ли формула сокращенного языка логики высказываний невыполнимой.

1. Грамматика сокращенного языка логики высказываний

<логическая константа> ::= 1|0

<символ> ::= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z

<отрицание> ::= !

<конъюнкция> ::= $\wedge$

<дизъюнкция> ::= $\vee$

<импликация> ::= $\rightarrow$

<эквиваленция> ::= $\sim$

<открывающая скобка> ::= (

<закрывающая скобка> ::=)

<атомарная формула> ::= <латинская заглавная буква>

<унарная сложная формула> ::= <открывающая
скобка><отрицание><формула><закрывающая скобка>

$\langle \text{бинарная сложная формула} \rangle ::= \langle \text{открывающая скобка} \rangle \langle \text{формула} \rangle$

$\langle \text{бинарная связка} \rangle \langle \text{формула} \rangle \langle \text{закрывающая скобка} \rangle$

$\langle \text{сложная формула} \rangle ::= \langle \text{унарная сложная формула} \rangle \mid \langle \text{бинарная сложная формула} \rangle$

$\langle \text{формула} \rangle ::= \langle \text{логическая константа} \rangle \mid \langle \text{атомарная формула} \rangle \mid \langle \text{сложная формула} \rangle$

2. Описание реализации

Для реализации был выбран язык C++ стандарта C++17.

2.1. Структуры данных

Для хранения структуры логической формулы использовалось бинарное несбалансированное дерево. Каждый узел дерева содержит в себе вид узла: (переменная, константа, отрицание, конъюнкция, дизъюнкция, импликация, эквиваленция), исходный фрагмент текста формулы, значение для константы, имя для переменной, индекс переменной в таблице истинности, а также указатели на левое и правое поддереву. В методах использовались структуры данных стандартной библиотеки: умные указатели для владения узлами дерева, ассоциативный массив для хранения интерпретации и индексов переменных, упорядоченное множество для накопления уникальных переменных, динамический массив и строка.

2.2. Алгоритм

Алгоритм начинает работу после передачи в метод `Parser::parse()` строки в качестве аргумента. Разбор строки происходит методом рекурсивного спуска и реализован в методе `Parser::parse_formula()` следующим образом:

- Чтение текущего символа исходной строки.

- Если символ является логической константой (0 или 1) - создание листового узла соответствующего вида.
- Если символ является заглавной латинской буквой - создание листового узла переменной, добавление переменной в множество уникальных переменных и присвоение ей порядкового индекса при первом появлении.
- Если символ является открывающей скобкой - определение типа сложной формулы:
 - Если следующий символ - отрицание (!), то выполняется рекурсивный разбор операнда и создается узел отрицания.
 - Иначе выполняется рекурсивный разбор левого операнда, затем чтение бинарного оператора ($\wedge$, $\vee$, $\rightarrow$, $\sim$), затем рекурсивный разбор правого операнда и создание узла соответствующего вида.
- Проверка наличия закрывающей скобки и сохранение исходного фрагмента текста в поле text созданного узла.

Для проверки логического выражения на выполнимость объект класса Node, представляющий корень дерева, передается в функцию построения таблицы истинности вместе с множеством переменных формулы.

Проверка происходит следующим образом:

- Формирование упорядоченного списка переменных из множества уникальных переменных формулы.
- Подсчёт количества строк таблицы истинности как 2 в степени количества переменных.
- Перебор всех возможных интерпретаций переменных в виде последовательного увеличения битовой маски от 0 до $2^n - 1$, где каждый бит маски соответствует значению одной переменной.
- Для каждой интерпретации - заполнение ассоциативного массива значений переменных и вычисление значения формулы методом `Node::eval()`, который рекурсивно обходит дерево и применяет операции в соответствии с видом каждого узла.

- Если на текущей интерпретации значение формулы равно истине, формула признается выполнимой и дальнейший перебор прекращается.
- Если ни одна интерпретация не дала значения «истина», формула признается невыполнимой.

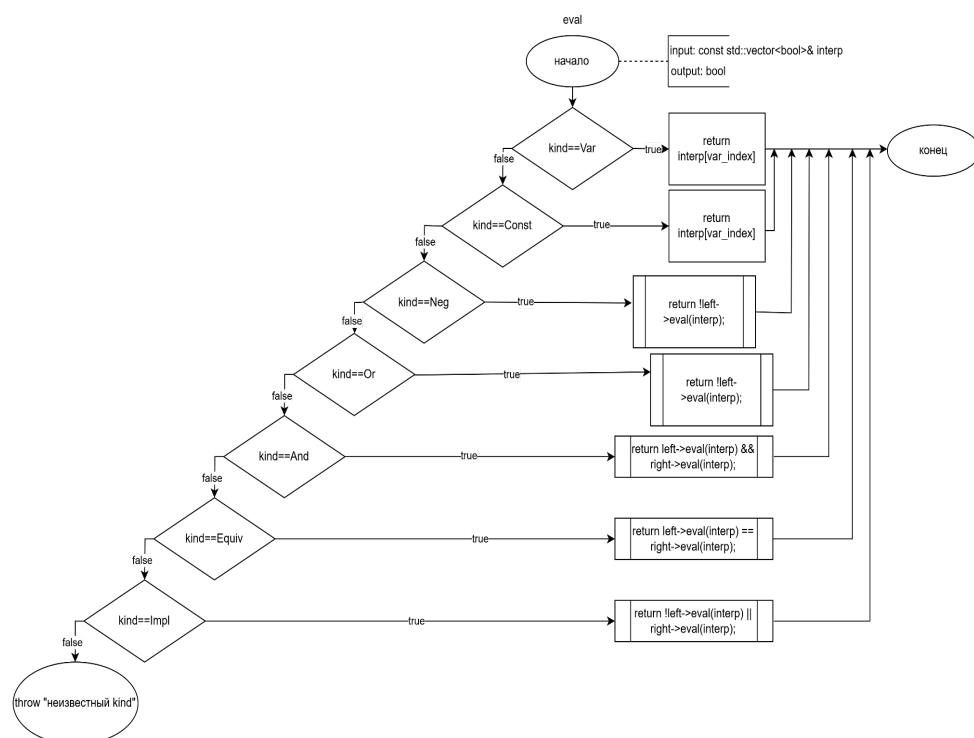
3. Схемы алгоритма

3.1 Схема алгоритма разбора строки

Данная схема поделена на отдельные схемы отдельных функций для большей читаемости

Функция `Node::eval()` определяет тип логической связки:

Рис. 3.1 - `Node::eval()`



Функция `Parser::current()` позволяет обратиться к текущему символу строки:

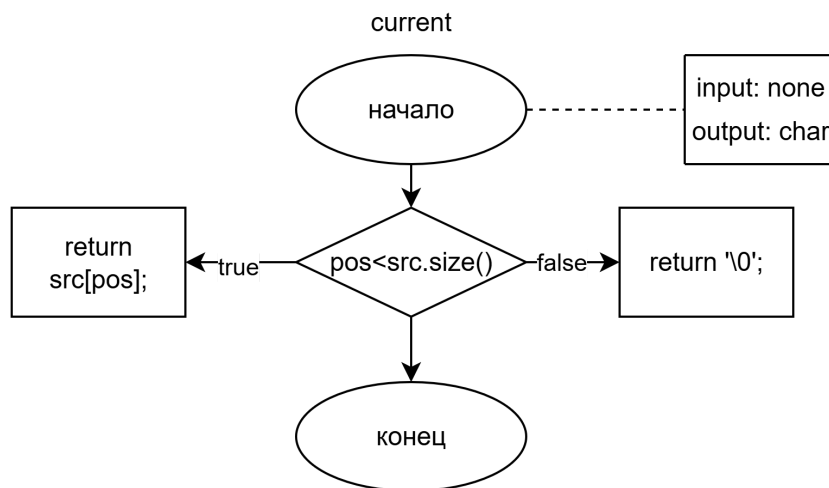


Рис. 3.2 - `Parser::current()`

Функция `Parser::move_symbol()` позволяет сдвинуться на символ вперед по строке:

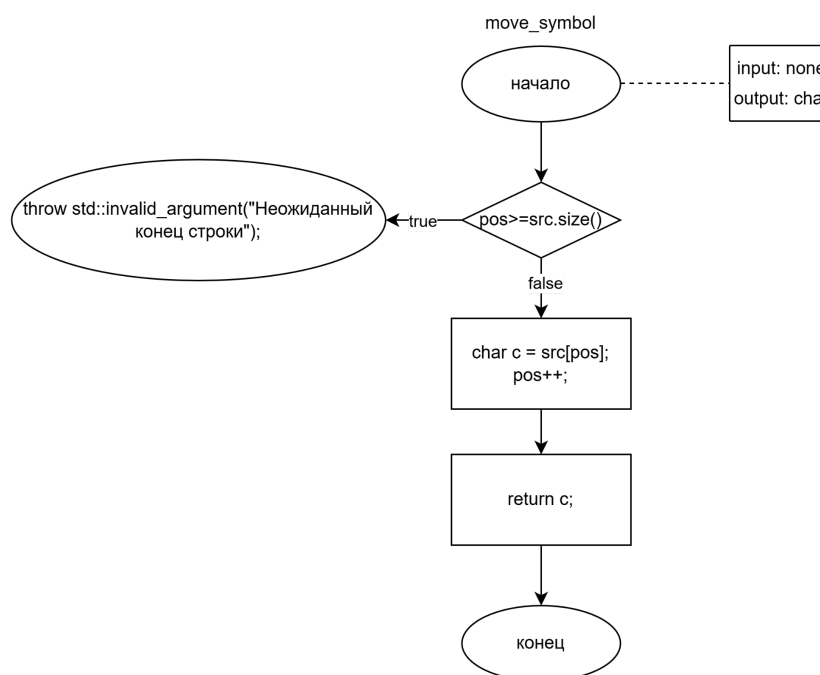


Рис. 3.3 - `Parser::move_symbol()`

Функция `Parser::expect()` ожидает на выходе определенный символ, если он не совпадает с ожидаемым - выбрасывается ошибка:

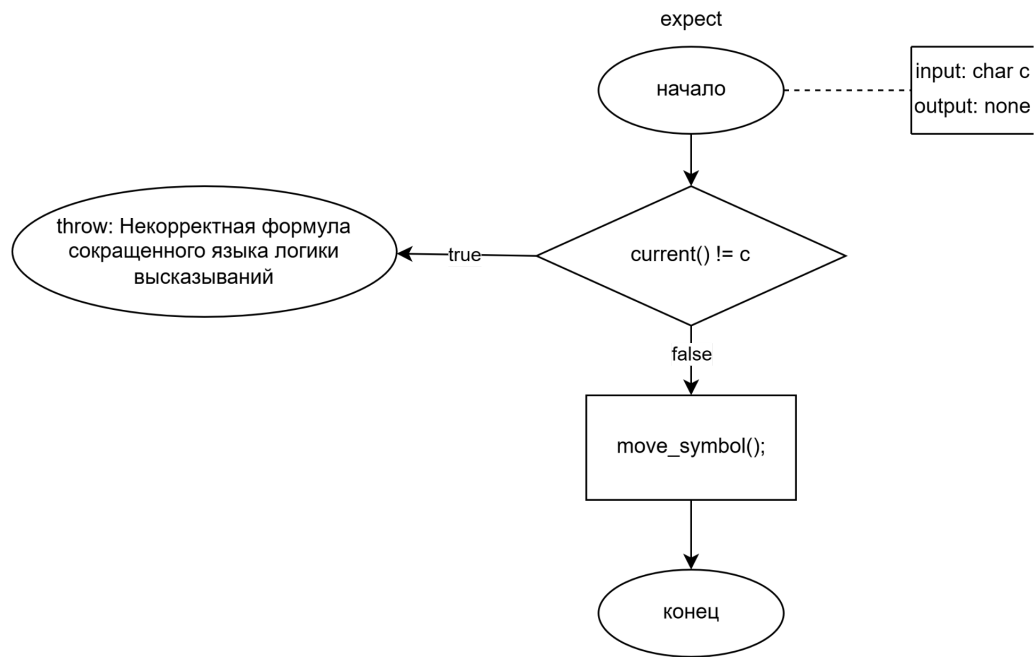


Рис. 3.4 - Parser::expect()

Функция Parser::parse() реализует обработку формулы сокращенного языка логики высказываний:

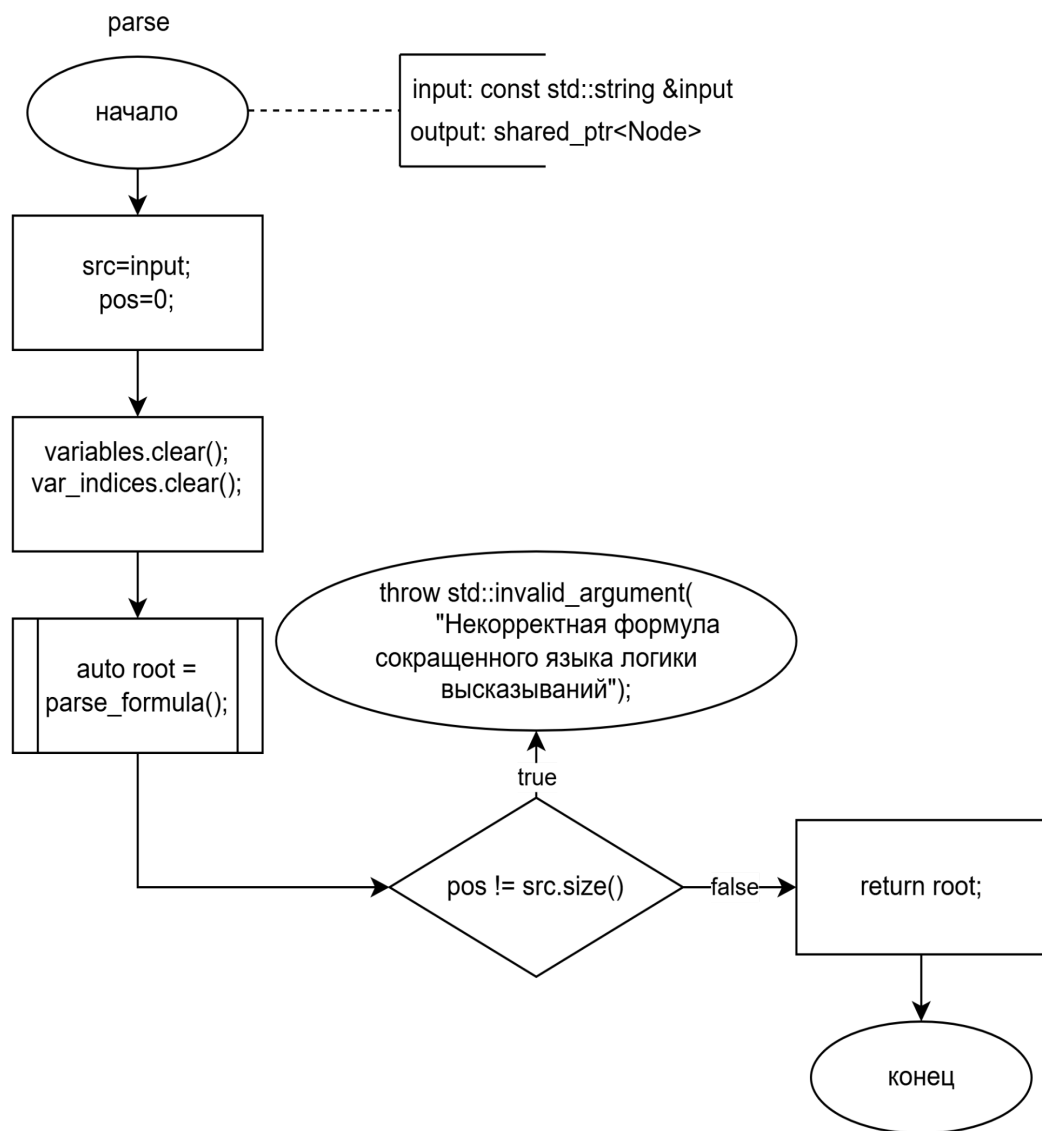
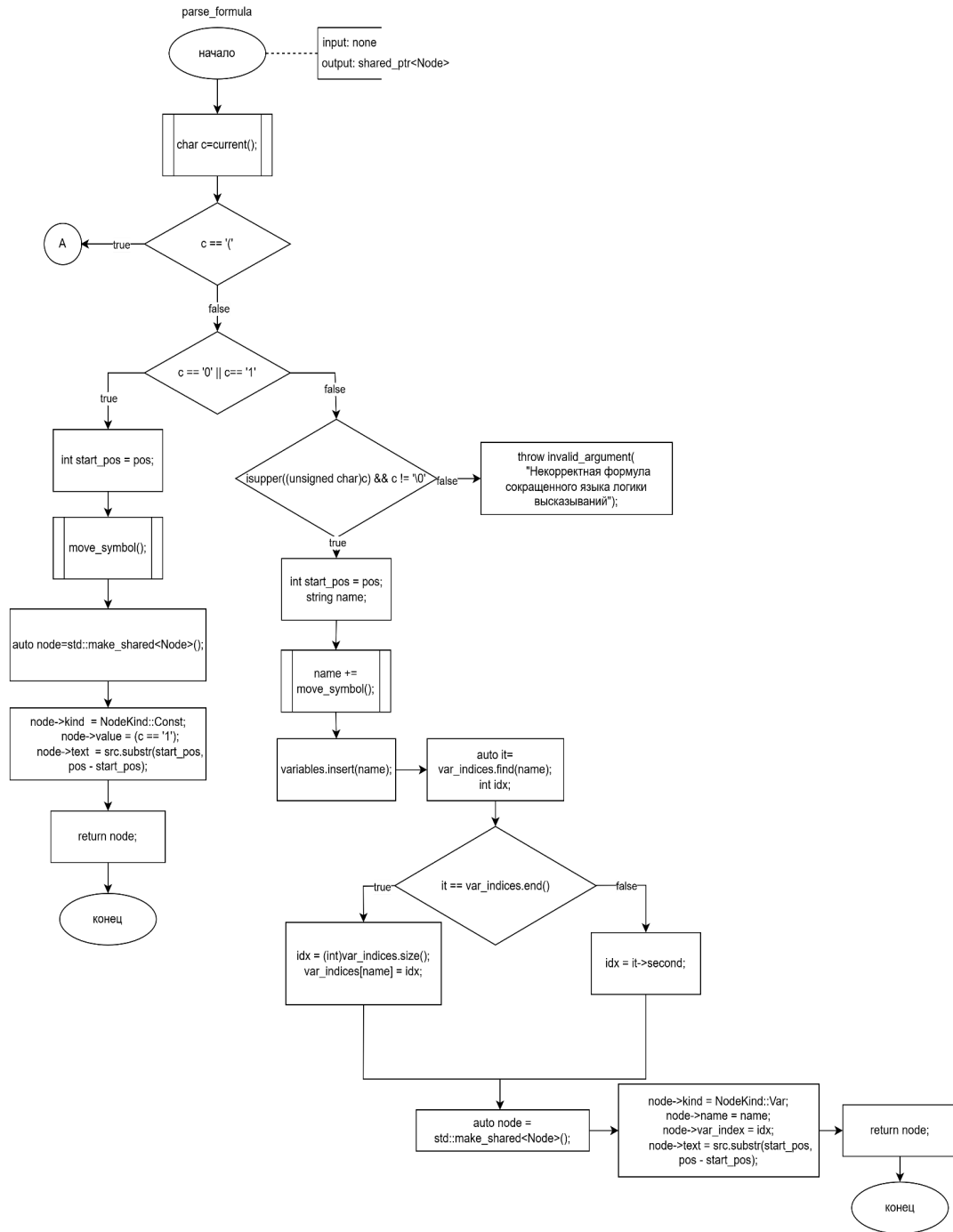
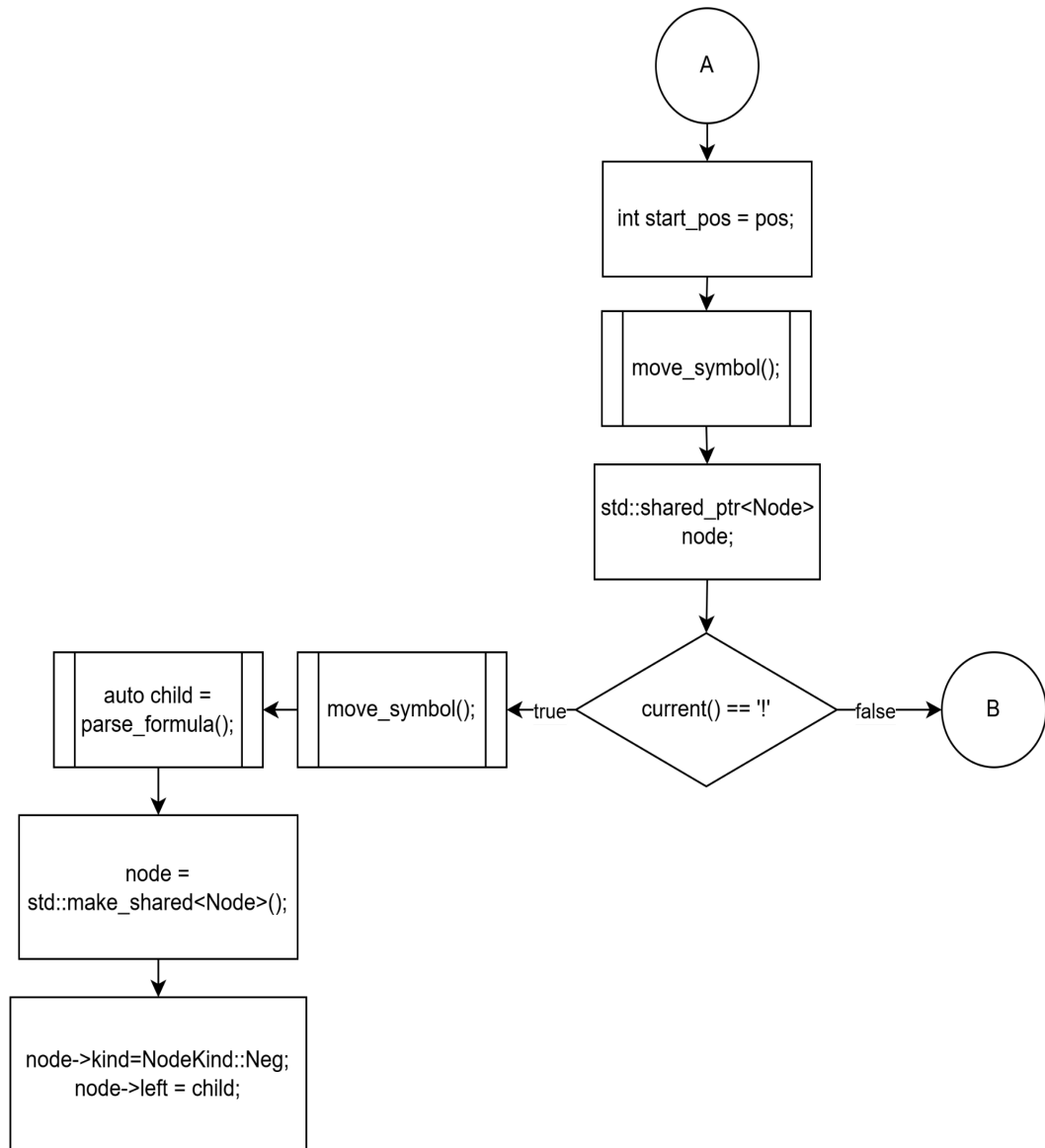


Рис. 3.5 - Parser::parse()

Функция Parser::parse_formula() позволяет построить дерево по формуле. Для удобства прочтения схема была поделена на несколько частей:





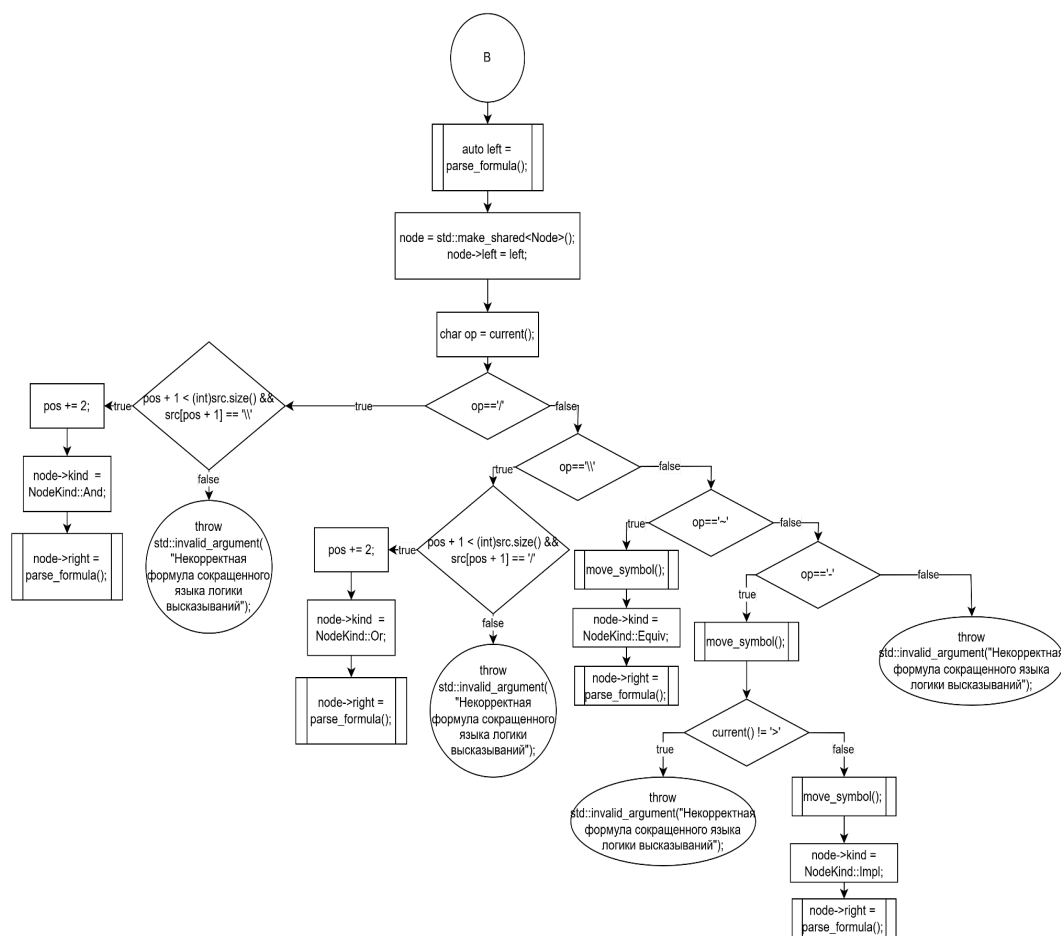


Рис. 3.6 - Parser::parse_formula()

Функция `build_truth_table()` позволяет строить таблицу истинности в зависимости от количества переменных. Если количество переменных в выражении больше обрабатываемого максимума, то пользователю сообщается, что таблица не выводится, а только идет проверка:

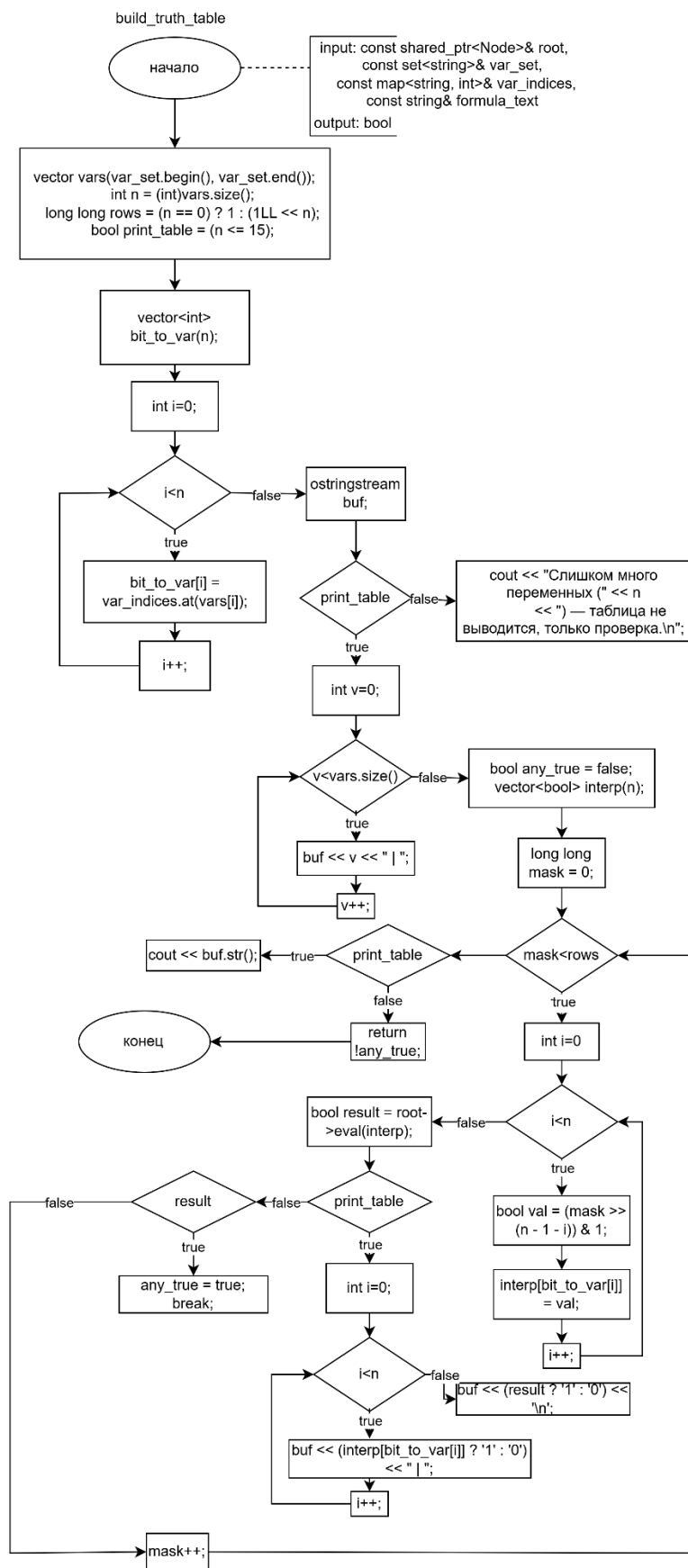


Рис. 3.7 - build_truth_table()

Функция `main()` является точкой входа программы и реализацией работы программы:

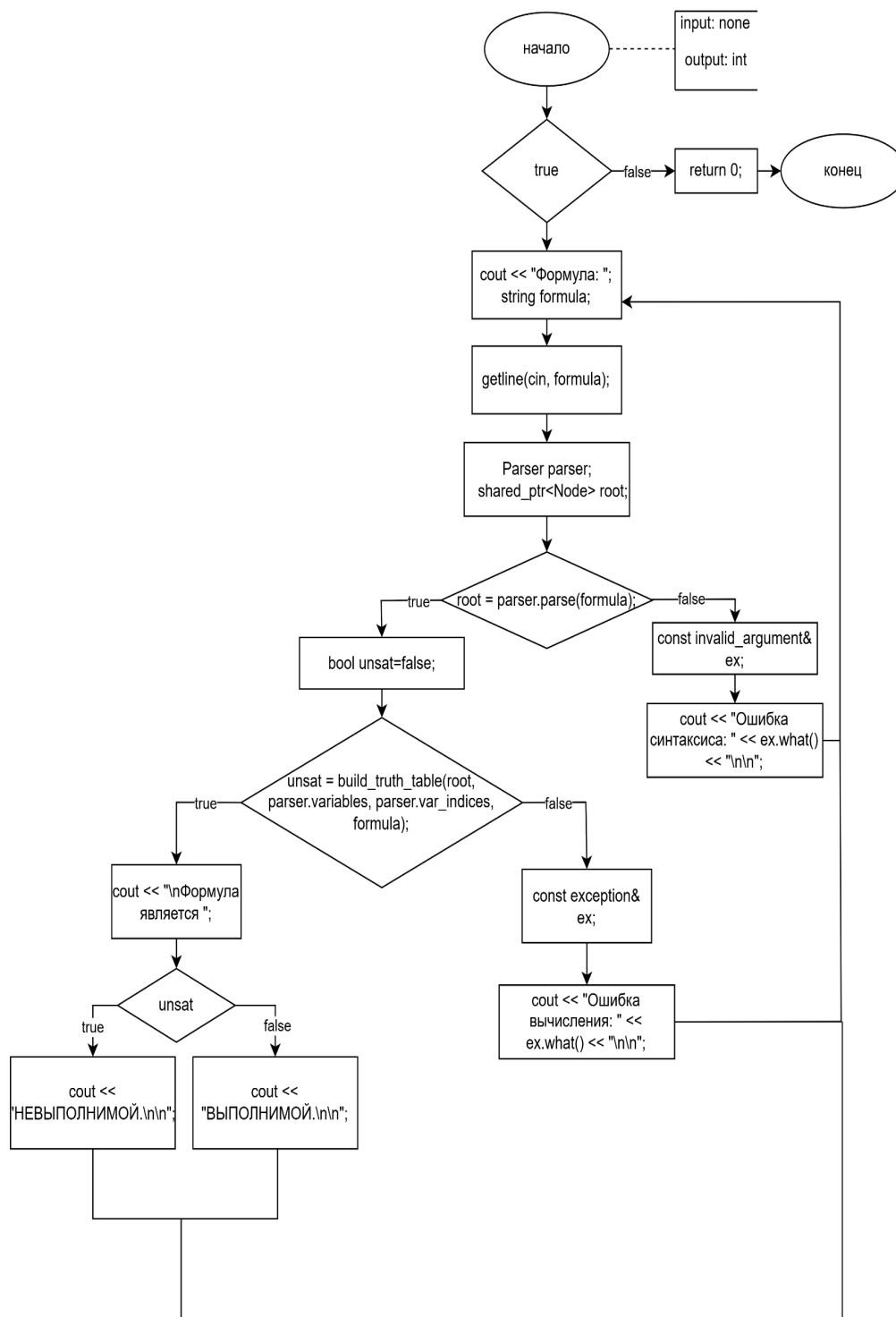


Рис. 3.8 - `main()`

4. Тестирование программы

При выполнении пользователя встречает окно с возможностью ввода формулы:

```
E:\LABS_FOURTH_SEM\LOIS\lab1\cmake-build-debug\lab1.exe
Формула:
```

В случае неверного ввода формулы будет выведено сообщение с ошибкой:

```
Формула: (!A

Ошибка синтаксиса: Некорректная формула сокращенного языка логики высказываний
```

Иначе программа начнет выполнение:

```
Формула: (A\B)

A | B | (A\B)
0 | 0 | 0
0 | 1 | 1
...

Формула является ВЫПОЛНИМОЙ.

Формула: (P\(!P))

P | (P\(!P))
0 | 0
1 | 0

Формула является НЕВЫПОЛНИМОЙ.
```

Максимальный предел количества переменных для построения в консоли таблицы истинности - 15. Для 15 переменных в худшем случае время работы программы составило 203 мс:

Тестовый случай:

$(A \wedge (B \wedge (C \wedge (D \wedge (E \wedge (F \wedge (G \wedge (H \wedge (I \wedge (J \wedge (K \wedge (L \wedge (M \wedge (N \wedge (O \wedge (!A)))))))))))))))))$

1	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1	0
1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0

Время выполнения: 203 мс

Формула является НЕВЫПОЛНИМОЙ.

При большом количестве переменных (>15), таблица истинности не строится.

Тестовый случай:

$(A \wedge (B \wedge (C \wedge (D \wedge (E \wedge (F \wedge (G \wedge (H \wedge (I \wedge (J \wedge (K \wedge (L \wedge (M \wedge (N \wedge (O \wedge (P \wedge (Q \wedge (R \wedge (S \wedge (T \wedge (!A))))))))))))))))))$

Результат:

Формула: $(A \wedge (B \wedge (C \wedge (D \wedge (E \wedge (F \wedge (G \wedge (H \wedge (I \wedge (J \wedge (K \wedge (L \wedge (M \wedge (N \wedge (O \wedge (P \wedge (Q \wedge (R \wedge (S \wedge (T \wedge (!A))))))))))))))))))$

Слишком много переменных (20) – таблица не выводится, только проверка.

Время выполнения: 490 мс

Время работы для 20 переменных составило 490 мс.

Тестовый случай:

$(A \rightarrow (B \wedge (C \rightarrow (D \wedge (!E) \rightarrow (F \vee (G \rightarrow (H \wedge (I \rightarrow (J \sim (K \rightarrow (L \wedge (!M) \rightarrow (N \rightarrow (O \wedge (!P) \vee (Q \sim (R \rightarrow (S \rightarrow (T \wedge (U \sim (V \rightarrow (W \rightarrow Z))))))))))))))))))))))$

Результат:

```
Формула: (A/\(B/\(C/\(D/\(E/\(F/\(G/\(H/\(I/\(J/\(K/\(L/\(M/\(N/\(O/\(P/\(Q/\(R/\(S/\(T/\(U/\(V/\(W/\(X/\(Y/\(Z))))))))))))))))))
Слишком много переменных (26) – таблица не выводится, только проверка.
Время выполнения: 65360 мс
```

Время работы для максимального возможного случая(26 переменных) составило 65360 мс.

Вывод

В результате выполнения лабораторной работы были приобретены навыки программирования алгоритмов синтаксического разбора формул сокращенного языка логики высказываний. Также была реализована процедурная программа синтаксического разбора формул сокращенного языка логики высказываний, проверяющая, является ли формула невыполнимой.

Список использованных источников

- Логические основы интеллектуальных систем. Практикум: учеб.-метод. пособие / В. В. Голенков [и др.]. - Минск: БГУИР, 2011.
- Метод рекурсивного спуска - URL: [https://ru.wikibooks.org/Реализации_алгоритмов/Метод рекурсивного спуска](https://ru.wikibooks.org/Реализации_алгоритмов/Метод_рекурсивного_спуска) (дата обращения: 03.04.2026)